COMPUTE PROVISIONING RUNBOOK — NX-OPS-KARP-005

Karpenter NodePools, Consolidation & Spot Strategy

Nexlify Technologies | Applies to nexlify-prod-eks & nexlify-nonprod-eks

1. Karpenter Deployment Overview

Karpenter v0.37+ is the sole node provisioning engine for both Nexlify EKS clusters. The classic Cluster Autoscaler has been removed. Karpenter controller runs in the platform-system namespace with a dedicated IRSA role that can launch EC2 instances, manage instance profiles, and terminate nodes. The controller is highly available (2 replicas) and watches the default and application namespaces for unschedulable pods. NodeClaims are created dynamically based on NodePool and EC2NodeClass definitions. All node lifecycle events are emitted as Kubernetes events and also forwarded to CloudWatch for cost and capacity analysis.

2. NodePool Design — Resource-Based Separation

Three primary NodePools are defined to isolate workload characteristics and optimise bin-packing:

- NodePool 'general-purpose' — matches pods that request CPU and memory without specialised hardware. Instance families: m6i, m6a, m7i, m7a. Weight 50. Consolidation policy: WhenUnderutilized. ExpireAfter: 720h. CPU and memory limits are set per NodePool to prevent runaway scaling.
- NodePool 'memory-optimised' — for pods with memory:cpu ratio > 4:1 (e.g., Redis, JVM heaps, in-memory caches). Instance families: r6i, r6a, r7i. Higher weight for memory-intensive pods via nodeAffinity. Consolidation still active but disruption budgets tighter during business hours.
- NodePool 'io-intensive' — for workloads that declare high ephemeral-storage or use local NVMe (e.g., ClickHouse, Kafka with local disks). Instance families: i4i, im4gn. Taints 'workload-type=io' ensure only tolerating pods land here. Consolidation is more conservative because data locality matters.

3. Spot Capacity & On-Demand Fallback

Non-production NodePools request Spot capacity exclusively (capacity-type: spot). Production NodePools use a mixed strategy: requirements include both spot and on-demand, with a weight preference for Spot. Karpenter's native Spot-to-on-demand fallback is enabled; when Spot capacity is unavailable for a given shape, the controller automatically launches on-demand instances of the same family. The EC2NodeClass defines a diversified list of instance types so that Spot interruption risk is spread. Interruption handling uses the Karpenter interruption queue (SQS) subscribed to EventBridge EC2 Spot interruption warnings; nodes are drained gracefully before reclamation. On-demand percentage target for production is approximately 30% steady-state; the remainder is Spot. Cost dashboards track the realised ratio weekly.

4. Consolidation & Disruption Controls

Consolidation is enabled on all NodePools with policy WhenUnderutilized. Karpenter continuously evaluates whether pods can be moved to fewer or cheaper nodes. Disruption budgets limit simultaneous voluntary disruptions: production allows max 10% of nodes or 2 nodes (whichever smaller) per 10-minute window during peak hours (09:00–21:00 IST); outside peak the budget is relaxed to 20%. Non-prod has no time-based restriction and allows 30% disruption. PodDisruptionBudgets of applications are respected; Karpenter will not violate a PDB. Expiration (ExpireAfter) forces periodic node recycling to pick up AMI and security updates; 30 days for non-prod, 14 days for production security-sensitive pools.

5. Operational Notes & Troubleshooting

When pods remain Pending with 'no instance type satisfies requirements', check NodePool requirements against the pod's requests, tolerations, and topology constraints. Common issues: AZ mismatch with PVC zone affinity, insufficient Spot capacity in a specific family, or a NodePool limit that has been hit. Karpenter logs are available via kubectl logs -n platform-system -l app.kubernetes.io/name=karpenter. The 'karpenter_nodes_allocatable' and 'karpenter_cloudprovider_instance_type_offering_available' metrics are the first place to look for capacity problems. For Redis and other stateful services that must stay in ap-south-1a, the NodePool topologySpreadConstraints and the pod's requiredDuringScheduling affinity are configured accordingly.

Owner: Compute Platform | Slack: #karpenter | Rev 3.3 | 2026-07-12